

Guía de consumo de APIs — UMG Personaliza

Entregable 1.1.2.5 del Proyecto Final Desarrollo Web 2027.

Todo lo que aparece aquí está **verificado contra el sistema en ejecución** (2026-09-02, actualizado el 2026-09-24): los códigos de respuesta, la matriz de permisos y los formatos de error se obtuvieron ejecutando las peticiones, no leyendo el código. La colección Postman se ejecuta completa con Newman: 86 peticiones y 128 aserciones, todas en verde.

La colección Postman lista para importar está en

[[UMG_Personaliza.postman_collection.json](#)](fuentes/UMG_Personaliza.postman_collection.json).

1. Dirección base y prefijos

Entorno	Base
Local con Docker	http://localhost:8081
Backend directo (sin Nginx)	http://localhost:3000

> En Windows el puerto 8081 puede estar en el rango de puertos excluidos del > sistema y el arranque falla con **bind: Intento de acceso a un socket no permitido**. > Compruébalo con **netsh interface ipv4 show excludedportrange protocol=tcp** y, si > aparece, publica el frontend en otro puerto mediante un **docker-compose.override.yml**.

Hay **tres prefijos distintos** y conviene tenerlos claros porque no son uniformes:

Prefijo	Qué agrupa	Ejemplo
(raíz)	Autenticación y registro	POST /login
/api/tienda	Catálogo, carrito, órdenes, entregas	GET /api/tienda/productos
/admin	Usuarios, roles, auditoría, reportes	GET /admin/usuarios

Advertencia: el login **no** cuelga de [/api/auth](#). Está en **POST** [/login](#). Las únicas rutas de autenticación con ese prefijo son **forgot-password** y **reset-password**.

2. Autenticación

Obtener el token

POST [/login](#)

Content-Type: application/json

```
{ "usuario": "test", "contrasena": "test123" }
```

Respuesta:

```
{
  "ok": true,
  "metodo": "password",
  "mensaje": "Login exitoso",
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "Bearer",
```

```
"usuario": { "id": 2, "usuario": "test", "rol": "Comprador", "correo": "..." }
}
```

Usar el token

```
GET /api/tienda/carrito
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Qué contiene el token

sub, uid, usuario, correo, rol, mfa, iat, exp, aud, iss.

- **Vigencia: 15 minutos.** Pasado ese tiempo hay que volver a autenticarse.
- Emisor `umg_personaliza`, audiencia `umg_personaliza_frontend`.
- El campo `mfa` indica con qué modo se autenticó: `password`, `facial` o `qr`.

Los tres modos de login usan el mismo endpoint

Esto es lo que más confunde al integrar. **No existen** `/api/auth/login-face` ni `/api/auth/login-qr`. El servidor decide el modo según qué campos recibe:

Modo	Campos que envías
Contraseña	usuario (o correo) + contraseña
Reconocimiento facial	usuario (o correo) + photo_base64
Código QR	qr, sin usuario ni correo

En el modo facial el servidor extrae un descriptor de 128 dimensiones y lo compara con el almacenado, con umbral de distancia 0.6. La imagen no puede superar unos 2.5 MB en base64 o responde **413**.

3. Permisos por rol

Matriz obtenida ejecutando cada endpoint con los cuatro roles. Los roles son Administrador (4), Supervisor (5), Comprador (6) y Repartidor (7).

Endpoint	Sin token	Comprador	Admin	Supervisor	Repartidor
GET /api/tienda/productos	200	200	200	200	200
GET /api/tienda/categorias	200	200	200	200	200
GET /api/tienda/areas	200	200	200	200	200
GET /api/tienda/carrito	401	200	200	403	403
GET /api/tienda/perfil	401	200	200	403	403
GET /api/tienda/ordenes/mis	401	200	200	403	403
GET /api/tienda/ordenes/admin/list	401	403	200	200	200
GET /api/tienda/ordenes/export/csv	401	403	200	200	200
GET /api/tienda/areas/admin	401	403	200	200	200
GET /api/tienda/dashboard/ventas	401	403	200	200	403
GET /api/tienda/productos/admin/list	401	403	200	403	403
GET /admin/usuarios	401	403	200	200	—
GET /admin/auditoria	401	403	200	200	—
GET /admin/access-logs	401	403	200	200	—
GET /admin/roles	401	403	200	403	—
GET /admin/__ping	200	200	200	200	200
GET /admin/roles-public	200	200	200	200	200

Dos detalles que sorprenden y conviene no descubrir depurando:

- El Supervisor no puede ver el carrito ni el perfil de comprador. Esas rutas son de Comprador y Administrador únicamente.
- `/admin/perfil/:id` solo devuelve tu propio perfil. Compara el `id` de la URL con el `uid` del token y responde 403 si no coinciden, aunque seas Administrador.

4. Los cuatro métodos

GET — consultar

GET `/api/tienda/productos`

```
{
  "ok": true,
  "productos": [
    {
      "id": 1, "nombre": "Llavero Clasico UMG", "slug": "llavero-clasico",
      "descripcion": "...", "precio": "35.00", "stock": 50,
      "imagen_url": null, "tiene_lado_b": true,
      "categoria": "Llaveros", "categoria_slug": "llaveros"
    }
  ]
}
```

POST — crear

POST `/api/tienda/carrito/items`

Authorization: Bearer <token de Comprador>

Content-Type: application/json

```
{
  "id_producto": 1,
  "cantidad": 2,
  "personalizacion": {
    "ladoA": { "texto": "Hola", "filtro": "ninguno" },
    "ladoB": { "texto": "UMG 2027" }
  }
}
```

La personalización de Lado A y Lado B viaja como objeto JSON dentro del campo `personalizacion`. El servidor la guarda en la columna `personalizacion_json`, pero en la petición el campo se llama `personalizacion`: si se envía con el otro nombre se ignora sin error y el artículo queda sin personalizar.

PUT — actualizar

PUT `/api/tienda/carrito/items/1`

Authorization: Bearer <token de Comprador>

Content-Type: application/json

```
{ "cantidad": 3 }
```

DELETE — eliminar

DELETE `/api/tienda/carrito/items/1`

Authorization: Bearer <token de Comprador>

Peticiones con archivos

POST /registro, POST /api/tienda/personalizacion/imagen, POST /api/tienda/ordenes/:id/entrega y POST /admin/perfil/foto usan multipart/form-data, no JSON. En Postman se configuran en la pestaña **Body** → **form-data**, marcando como **File** los campos de imagen.

5. Formato de errores

Todas las respuestas de error devuelven JSON. Nunca exponen detalles de la base de datos ni trazas de pila.

Código	Cuándo	Cuerpo
400	Datos inválidos o incompletos	{ "error": "..." }
401	Falta el token o expiró	{ "error": "Token requerido", "code": "no_token" }
403	Token válido pero rol sin permiso	{ "error": "No tienes permisos para esta accion", "code": "forbidden" }
404	El recurso no existe	{ "error": "Guía de rastreo no encontrada." }
409	Conflicto con el estado actual: transición de orden no permitida, entrega de una orden que no salió del taller o slug de producto repetido	{ "error": "...", "estadoActual": "..." }
413	Imagen demasiado grande en login facial	{ "error": "Imagen demasiado grande..." }
429	Se superó el límite de peticiones	{ "error": "Demasiados intentos de acceso..." }
500	Error del servidor	{ "ok": false, "error": "..." }
503	Pago con tarjeta sin claves de Recurrente configuradas	{ "error": "..." }

Un login con credenciales incorrectas devuelve { "error": "Usuario o contraseña inválidos" } **sin distinguir** si falló el usuario o la contraseña, para no facilitar la enumeración de cuentas.

Límites de peticiones

Endpoint	Límite
POST /login	25 intentos por IP cada 5 minutos
POST /api/tienda/checkout	15 peticiones cada 2 minutos

Al superarlos responde 429 indicando cuántos segundos hay que esperar.

6. Flujo completo de compra

Orden en que se encadenan las llamadas:

1. POST /login → guarda el token 2. GET /api/tienda/productos → elegir artículo 3. POST /api/tienda/carrito/items → agregar con su personalización 4. GET /api/tienda/areas → elegir punto de entrega del campus 5. POST /api/tienda/checkout → genera la orden

```
{ "id_area_entrega": 1, "notas_entrega": "Dejar en recepción",  
  "metodo_pago": "efectivo", "recaptcha_token": "" }
```

Devuelve el código UMG-XXXX, deja la orden en estado **recibida**, emite la constancia PDF con QR y publica el estado por WebSocket.

6. GET /api/tienda/ordenes/rastreo/{codigo} → seguimiento (público, sin token) 7. GET /api/tienda/ordenes/{codigo}/constancia → descargar el PDF

metodo_pago admite efectivo o tarjeta. Con tarjeta el servidor crea un checkout en la pasarela Recurrente y la respuesta trae orden.url_pago, la página donde el comprador introduce su tarjeta (el número nunca pasa por este servidor). Mientras RECURRENTE_SECRET_KEY esté vacía, el pago con tarjeta responde 503.

Desde su creación, la orden avanza sola: pasa a en_elaboracion a los 15 s y a lista_entrega a los 60 s (variables ELABORACION_* del .env).

recaptcha_token puede ir vacío mientras RECAPTCHA_SKIP=true en el entorno.

7. Cambios de estado de una orden

PUT /api/tienda/ordenes/:id/estado está gobernado por el autómata de backend/utils/ordenAutomata.js. Solo acepta transiciones legales:

Estado actual	Puede pasar a
recibida	en_elaboracion, cancelada
en_elaboracion	en_ruta, lista_entrega, cancelada
en_ruta	entregada, no_encontrado, cancelada
lista_entrega	entregada, no_encontrado, cancelada
no_encontrado	en_ruta, cancelada
entregada	— estado final
cancelada	— estado final

Cualquier otra combinación devuelve 409 con las transiciones válidas desde el estado actual. Además valida el rol: un Comprador no puede mover una orden a entregada.

POST /api/tienda/ordenes/:id/entrega (repartidor) solo acepta órdenes en en_ruta, lista_entrega o no_encontrado; en cualquier otro estado responde 409.

8. Seguimiento en tiempo real (WebSocket)

El tracking no va por HTTP sino por WebSocket:

```
ws://localhost:8081/ws/tienda?codigo=UMG-XXXX
```

El servidor emite un mensaje cada vez que la orden cambia de estado. Postman no cubre este canal en la colección; para probarlo sirve la vista /comprador/tracking.html o cualquier cliente WebSocket.

9. Repartidores y reportes

GET /admin/conductores, GET /admin/conductores/:id/historial y POST /admin/reportes/export con tipo: "conductores" devuelven los usuarios con rol Repartidor, su estado y su actividad (último acceso y número de ingresos, tomados de la bitácora access_logs). El reporte se genera en xlsx (por defecto) o pdf con el campo formato.

Hasta el 2026-09-24 estos endpoints respondían 500 porque consultaban una tabla conductores que no existe en el esquema.